

AI-Driven Public Health Chatbot for Disease Awareness

Project Proposal for UCS503P

Submitted to: Dr. Nisha Thakur

Thapar Institute of Engineering and Technology

Vyom Gupta, CSED*

Mani Gemini, CSED[†]

Vasu Garg, CSED*

Ariha, CSED[†]

August 19, 2026

1 Higher-order goal

This project aims to improve public health outcomes by reducing the population's reliance on unverified, informally circulated health information (e.g., WhatsApp forwards, generic web searches). While the topic of health misinformation is broader than any single application, the immediate engineering objective is to translate safe, verified disease-awareness delivery into a measurable, deployable software solution.

2 Time-to-value

- We will deliver meaningful increments quickly by prototyping the core retrieval – augmented generation (RAG) pipeline on a small, verified knowledge base and validating it against sample queries within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations (and CI-backed automation introduced early) to reduce release friction.
- Success is defined by what can be delivered and measured in the first iteration – a working grounded-answer pipeline on a limited disease set – then improved based on evaluation results.

3 Problem Statement

Access to reliable, verified health information remains limited for large sections of the population, especially outside major cities. In the absence of immediate access to a doctor, people often turn to unverified sources for symptom and disease information. Common pain points include:

- Misinformation risk: unverified WhatsApp forwards and generic web searches frequently contain inaccurate or misleading health claims.

- **Hallucination risk in plain AI chatbots:** a chatbot connected directly to a large language model can state incorrect medical information fluently and with full confidence.
- **No safe emergency handling:** general-purpose chatbots are not designed to recognize emergency-indicating language and escalate appropriately — they may continue a normal conversation when a user needs urgent care.
- **Scope ambiguity:** existing tools often blur the line between health education and medical diagnosis, creating risk of over-reliance on unverified AI output for actual medical decisions

Why this matters now (contextual relevance): With AI chatbots increasingly used as a first point of contact for health questions, even small failures in grounding or emergency detection can have real safety consequences. A system engineered specifically to avoid these failure modes is both timely and necessary.

4 Proposed Solution

4.1 Overview

Build a **Retrieval-Augmented Generation** (RAG) based chatbot with the following components:

- **Knowledge base builder:** collects and cleans verified disease information from trusted public sources (WHO, ICMR, National Health Portal, MoHFW advisories) and chunks it for retrieval.
- **Embedding & vector store:** converts text chunks into embeddings and stores them in a vector database (ChromaDB) for semantic search.
- **Retrieval + LLM response engine:** the most relevant verified chunks for a query and generates a grounded answer via an LLM API, explicitly instructed to answer only from supplied context.
- **Safety & escalation layer:** detects emergency-indicating language before any AI generation begins, and redirects the user to professional help instead of continuing the conversation
- **Chat frontend:** a user-facing interface for submitting questions and viewing grounded, source-noted responses.

4.2 Core workflow

1. The user submits a health-related question through the chat interface.
2. The message is scanned for emergency-indicating language (e.g. chest pain, breathing difficulty, suicidal ideation) before any AI generation begins.
3. If flagged, the pipeline stops immediately and returns a fixed escalation message directing the user to seek urgent medical help — no AI generation occurs on this path.
4. If not flagged, the query is embedded and compared against the vector database; the most relevant verified chunks are retrieved.
5. The retrieved chunks, the user's question, and explicit grounding instructions are sent to the LLM, which must answer using only the supplied verified context, or state that no verified information was found.
6. The grounded answer is returned to the user, optionally noting the source it was drawn from.

4.3 Operational constraints for an educational, setting

- Keep the knowledge base scoped to 8–10 common, well-documented diseases (e.g. dengue, malaria, typhoid, diabetes, hypertension, tuberculosis, chikungunya, common influenza) to keep the project achievable and verifiable within the course timeline.
- Avoid dependence on fine-tuned or custom-trained models; use -the-shelf embedding and LLM APIs, focusing engineering effort on grounding, safety, and correctness rather than model research.
- Design for deployability using common web hosting and managed/local services (no proprietary infrastructure required).

5 Solution Approach

This is an engineering course project, so we focus on building a reliable, safety-conscious retrieval pipeline rather than novel algorithm or model research.

5.1 Grounded retrieval pipeline

- Use standard, well-established techniques: fixed-size passage chunking (200–300 words), off-the-shelf embedding models, and cosine-similarity vector search — without claiming state-of-the-art novelty.
- Grounding is enforced through prompt design: the LLM is explicitly instructed to answer only from retrieved context, and to state clearly when no verified information is available rather than guessing.
- Emergency detection is implemented as a separate, deterministic keyword/phrase-matching layer that runs before any LLM call, so it can never be bypassed by the language model's own behaviour.

5.2 Web architecture

A typical 3-tier web architecture:

- **Frontend:** a lightweight chat interface (HTML, CSS, JavaScript) for submitting questions and viewing grounded responses.
- **Backend API:** FastAPI service handling query intake, safety screening, retrieval orchestration (via LangChain), and LLM calls.
- **Database:** ChromaDB storing chunk embeddings generated from the verified knowledge base.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run tests (including retrieval and escalation unit tests) on every change.
- Produce repeatable deployment artifacts to a staging environment.
- Enable safe and frequent releases of incremental features (knowledge base expansion, escalation refinement, etc.).

6 Evaluation Criterion

We define success using measurable metrics tied to the two core safety-relevant behaviours of the

system: grounding correctness and emergency escalation.

6.1 Primary evaluation metric

Emergency Escalation Accuracy (EEA): the percentage of emergency-indicating test queries — including paraphrased/ambiguous ones — that are correctly routed to the escalation message rather than a conversational answer.

- **Target:** 100% correct escalation on the clear-emergency test subset (zero tolerance for missed escalation on unambiguous cases).
- **Attribution:** measured by running the fixed 25–30 query evaluation set through the pipeline and logging the path taken (escalation vs. generation) for each query.

6.2 Secondary evaluation metrics

- **Grounding accuracy:** percentage of generated answers whose claims are verifiably drawn from the retrieved source chunks rather than fabricated, checked manually against the retrieved context.
- **Appropriate refusal rate:** percentage of out-of-scope or unsupported queries where the system correctly states that no verified information was found, instead of guessing.
- **Response latency:** end-to-end response time per query, targeted to remain within a few seconds.
- **User clarity:** reviewer-reported clarity of responses using a simple 1–5 feedback question.

6.3 Pilot validation plan

- A test set of 25–30 sample queries will be constructed, covering three categories: clearly non-emergency disease questions, clearly emergency scenarios, and ambiguous/paraphrased emergency language.
- Each query will be run through the pipeline and checked for (a) correct escalation behaviour and (b) whether generated answers are actually grounded in the retrieved source text. A small group of 3–5 peer reviewers will additionally rate response clarity and tone.

7 Scalability

The proposition should scale in both theoretical and practical senses.

- **Scalable (theoretically):** the system should support growth in the number of covered diseases and concurrent user queries by:
 - decoupling the knowledge base builder (offline) from the runtime query pipeline (online),
 - using a vector store that supports incremental addition of new chunks without full re-indexing,
 - designing a stateless backend API for horizontal scaling.
- 2. **Operationally deployable (practically):** The system should run on common web hosting:
 - managed or local vector database (ChromaDB),
 - platform-hosted backend (Render/Railway) and frontend (Vercel/Netlify) deployment,

- CI/CD pipeline for staging/production releases.

8 Engine availability heuristic

A mature set of “engines” (tooling/services) is available to implement this as a web-based project:

- Web framework for the backend REST API (FastAPI).
- RAG orchestration framework (LangChain) for chaining retrieval and generation steps.
- Vector database (ChromaDB) semantic search over the knowledge base.
- Embedding service (OpenAI Embeddings API, or sentence -transformers as a free local alternative).
- LLM API (Grok or OpenAI) for grounded response generation.
- Test framework for backend unit and integration tests.
- CI runner and staging deployment target.

We will use these mature components to focus on grounding correctness, safety-escalation design, and evaluation rather than inventing new infrastructure.

9 Project scope and deliverables

9.1 Initial deliverable

- Knowledge base builder covering 8–10 diseases source collection and text chunking (Module A).
- Embedding generation and vector store population (Module B).
- Basic retrieval + grounded-generation: with a tested grounding prompt (Module C).
- First-pass safety and escalation keyword layer (Module D).

9.2 CI: build + backend unit tests + linting. Subsequent deliverables

- Chat frontend (Module E) end-to-end to the backend.
- Paraphrase-robust escalation detection, extending beyond fixed keyword matching.
- Source citation display in the chatbot's responses.
- Formal evaluation against the 25–30 query test set, with documented results.
- Expanded knowledge base coverage and improved quality (e.g. re-ranking retrieved chunks).

10 Risks and mitigations

- **Residual hallucination risk:** with RAG, the LLM may occasionally deviate from retrieved context. Mitigated via a strict grounding prompt and an explicit “I don't have verified information on this — please consult a doctor” fallback instead of guessing.
- **Missed emergency detection:** paraphrased or indirect emergency language may bypass simple keyword matching. Mitigated via a layered detection approach, erring on

the side of caution, and explicitly evaluated using ambiguous test cases in the pilot validation plan.

- **Data privacy:** minimise collected user data; do not store personally identifiable information from chat sessions.
- **Source reliability drift:** medical guidance can change over time. Mitigated by sourcing only from WHO, ICMR, NHP, and MoHFW, with a periodic knowledge-base refresh process.
- **Team learning curve:** the team is new to RAG/LangChain tooling. Mitigated by dedicating the first two weeks explicitly to learning fundamentals and collecting source material before development begins.
- **Operational issues in pilot:** mitigated by using a staging environment early and ensuring CI/CD produces repeatable, testable deployments before any user-facing demo.

11 Summary

This project proposes a deployable, RAG-based chatbot to reduce reliance on unverified health information, while explicitly avoiding medical diagnosis and safely escalating emergency situations. It meets the course criteria by emphasizing time-to-value through rapid iterations, implementing CI/CD to support frequent safe releases, and using measurable, attributable evaluation criteria — particularly emergency escalation accuracy and grounding correctness. Its core technical contribution is not the language model itself, but the retrieval-grounding and safety-escalation layers engineered around it, which is the standard, responsible approach to deploying generative AI in a sensitive, safety-relevant domain.